Projeto de Arquitetura de Software - PSoft-P1

Projeto de Arquitetura de Software

Título:

Projeto e Documentação Arquitetural de um Sistema de Gestão de Livros

Contexto:

Arquitetura de Software

Docente:

Luís Henrique Ferreira

Autores:

Tomás Lopes - 1211289

Tiago Pinto - 1250552

Ano letivo:

2025/2026

Data:

2 de novembro de 2025

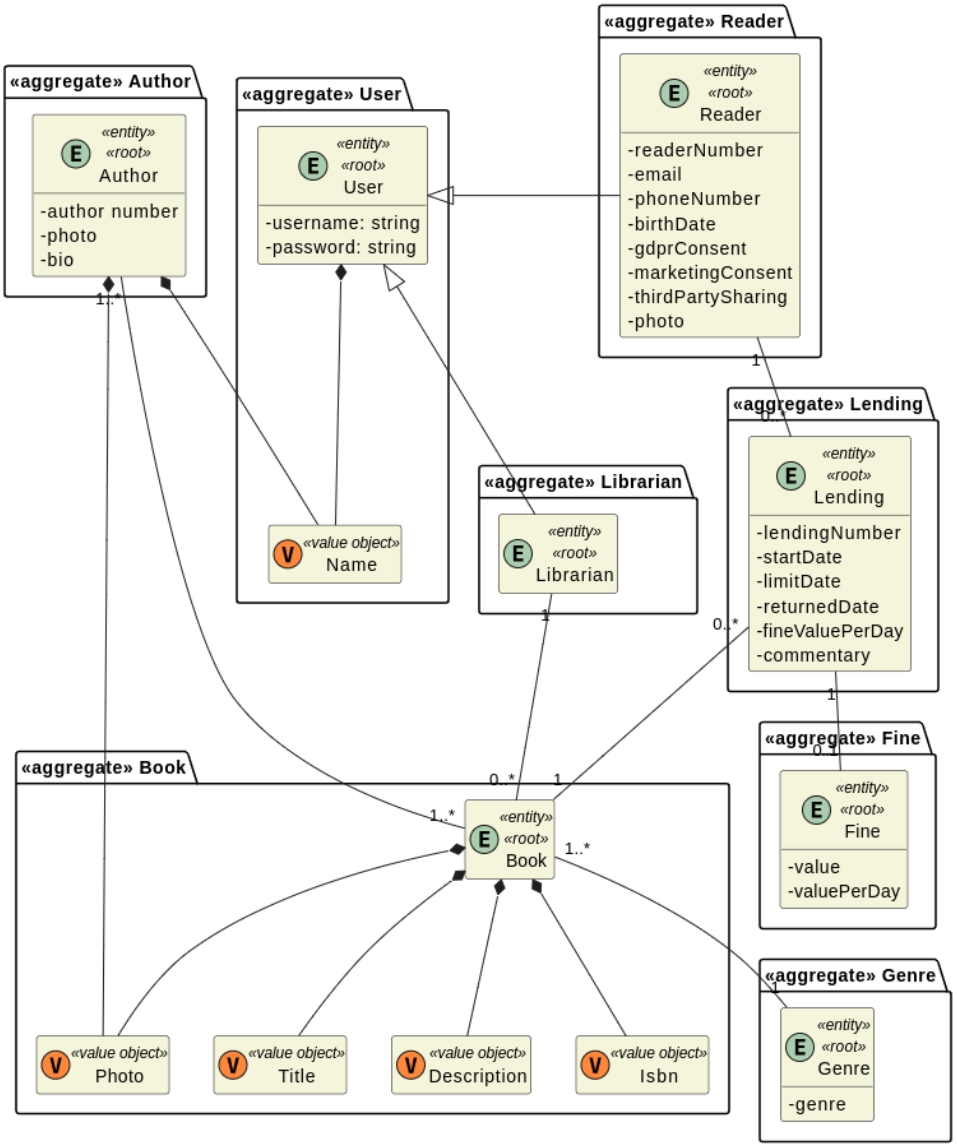
Library Management System - Documentation

System-as-is

1. System Overview

O sistema atual é um serviço backend orientado a REST para a gestão de uma biblioteca, desenvolvido em Java com Spring Boot e Maven. Permite a gestão de livros, géneros, autores, leitores e empréstimos, suportando operações de registo, consulta, atualização, pesquisa e estatísticas. A autenticação é feita via JWT e os dados são persistidos numa base de dados relacional (PostgreSQL/MySQL).

2. Domain Model



Entidades:

- **Livros:** Registo, atualização, consulta, pesquisa (por género, autor, título), foto de capa, estatísticas (top 5).

- **Géneros:** Registo, consulta, estatísticas.
- **Autores:** Registo, atualização, consulta, pesquisa, coautores, foto, estatísticas (top 5).
- **Leitores:** Registo (com foto e interesses), atualização, consulta, pesquisa (por nome, telefone, email), sugestões de livros.
- **Empréstimos:** Registo de empréstimos, devoluções, consulta, listagem de atrasos, estatísticas (duração média, top 5, por género/mês).
- **Multas (Fine):** Cálculo e registo de multas associadas a empréstimos em atraso, consulta de valores e histórico de multas.
- **Bibliotecários (Librarian):** Gestão de utilizadores com permissões administrativas, registo, autenticação e atribuição de funções.

Algumas das Relações:

- Um Livro pertence a um Género e tem um Autor.
- Um Empréstimo associa um Livro a um Leitor.
- Um Leitor pode ter vários Empréstimos.
- Fines estão associados a Empréstimos em atraso.

3. System Architecture (As-Is)

A arquitetura do sistema segue uma abordagem em camadas, típica de aplicações empresariais desenvolvidas com Spring Boot. Esta estrutura promove a separação de responsabilidades, facilitando a manutenção, a extensibilidade e a testabilidade do sistema.

O sistema está organizado nos seguintes componentes principais:

- **Controllers:**

  Responsáveis por tratar os pedidos HTTP recebidos e devolver as respostas adequadas. Cada controller implementa a lógica de interação entre a camada de routing e a camada de serviço. Os endpoints (por exemplo, `/api/books`, `/api/authors`, `/api/genres`, `/api/readers` e `/api/lendings`) são definidos através de anotações de mapeamento (Spring Boot), mas a responsabilidade do controller é exclusivamente orquestrar a comunicação entre o cliente e a lógica de negócio.

- **Services:**

  Implementam a lógica de negócio, incluindo regras de validação, orquestração de operações e coordenação entre diferentes componentes. Os serviços são responsáveis por garantir a integridade dos processos de negócio e por aplicar as regras específicas do domínio da biblioteca.

- **Repositories:**

  São interfaces que abstraem o acesso aos dados persistentes, utilizando Spring Data JPA para interagir com a base de dados relacional. Os repositórios permitem realizar operações de leitura e escrita sobre as entidades do domínio, sem expor detalhes de implementação.

- **Modelo de Domínio:**

  Inclui as entidades e value objects que representam os conceitos centrais do sistema, como Livro, Autor, Leitor, Empréstimo, Género, entre outros. O modelo de domínio está detalhado na secção anterior e serve de base para a persistência e lógica de negócio.

- **Segurança:**

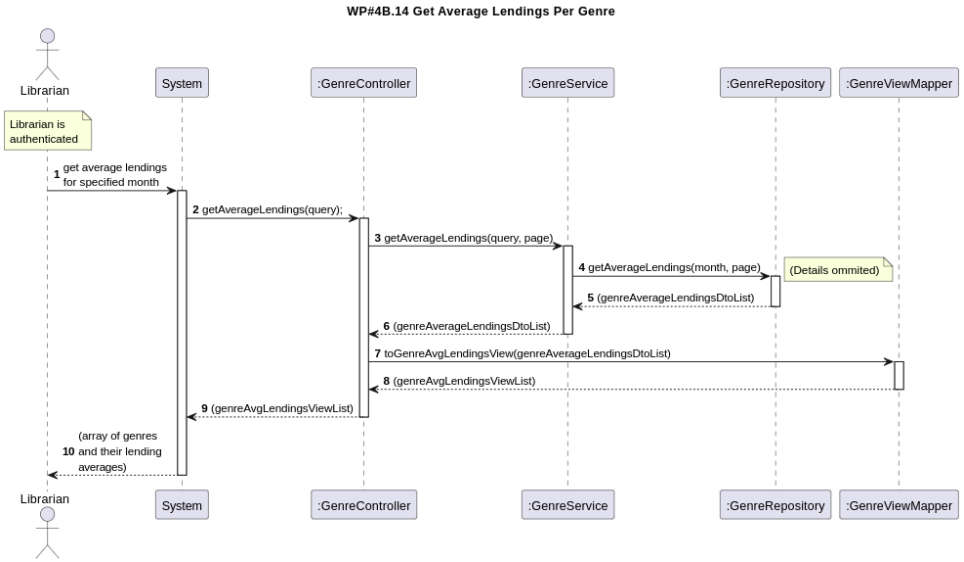
  A segurança do sistema é assegurada através da implementação de autenticação baseada em JWT (JSON Web Token), protegendo os endpoints sensíveis e garantindo que apenas utilizadores autenticados/autorizados podem aceder a determinadas funcionalidades.

- **Base de Dados:**

  O sistema utiliza uma base de dados relacional, com mapeamento das entidades através de JPA. Toda a informação relevante é armazenada de forma estruturada, garantindo integridade e consistência dos dados.

4. Exemplo de fluxo de Use Case

A Figura apresenta o diagrama de sequência para o caso de uso **"Obter Empréstimos Médios por Género"** (Get Average Lendings Per Genre). Segue-se uma explicação detalhada do fluxo:



1. O bibliotecário, já autenticado, solicita ao sistema os empréstimos médios por género para um determinado mês.
2. O `GenreController` recebe o pedido e invoca o método `getAverageLendings`.
3. O `GenreService` processa o pedido, chamando o método correspondente e passando os parâmetros necessários.
4. O `GenreService` solicita ao `GenreRepository` os dados dos empréstimos médios por género para o mês e página especificados.
5. O `GenreRepository` devolve uma lista de DTOs com os dados agregados.
6. O `GenreService` recebe a lista de DTOs e chama o `GenreViewMapper` para converter os dados para o formato de apresentação.
7. O `GenreViewMapper` transforma os DTOs numa lista de views.
8. O resultado é devolvido ao `GenreController`.
9. O `GenreController` responde ao sistema com a lista de géneros e respetivas médias de empréstimos.
10. O bibliotecário recebe um array de géneros e os seus valores médios de empréstimos para o mês especificado.

5. Requisitos Não Funcionais

O sistema já implementa um conjunto de requisitos não funcionais essenciais para garantir a segurança, a qualidade e a manutenibilidade da aplicação. Entre os principais aspetos já assegurados destacam-se:

- **Autenticação JWT:**

  Todos os endpoints sensíveis estão protegidos por autenticação baseada em JSON Web Tokens, garantindo que apenas utilizadores autorizados podem aceder a funcionalidades restritas.

- **Documentação OpenAPI/Swagger:**

  A API está devidamente documentada utilizando OpenAPI/Swagger, facilitando a compreensão, integração e teste dos endpoints por parte de programadores e utilizadores externos.

- **Coleção Postman:**

  Existe uma coleção Postman (Docs/Psoft-G1.postman_collection.json) que permite testar facilmente todos os endpoints da API, promovendo a validação e a automação de testes.

- **Estrutura Modular:**

O sistema apresenta uma arquitetura modular, com separação clara de responsabilidades entre controllers, serviços, repositórios e entidades, facilitando a manutenção e a evolução do código.

- **Testes Automatizados:**

  Foram desenvolvidos testes automatizados para os principais casos de uso, assegurando a robustez e a fiabilidade das funcionalidades implementadas.

Requisitos, especialmente ASR e Classificação

ID	Tipo	Requisito	Descrição	Atributo de Qualidade Afetado	Racional / Impacto Arquitetural	Prioridade
ASR-01	Funcional	Suporte a múltiplos modelos de persistência	O sistema deve permitir guardar dados em diferentes tipos de bases de dados: (i) SQL + Redis, (ii) MongoDB + Redis	Extensibilidade, Configurabilidade, Desempenho	Implica o desacoplamento da camada de persistência e o uso de abstrações ou padrões como *Repository* e *Strategy* para permitir a troca dinâmica de SGBDs.	Alta
ASR-02	Funcional	Configuração dinâmica da persistência em tempo de execução	O sistema deve permitir selecionar o modelo de persistência antes da execução	Configurabilidade, Manutenibilidade	Requer suporte a carregamento de configuração (ex: Spring Profiles, ficheiros de configuração) e uso do padrão Factory para instanciar as implementações adequadas.	Alta
ASR-03	Funcional	Integração com serviços externos de ISBN	O sistema deve obter o ISBN dos livros a partir de APIs externas: ISBNdb, Google Books API e Open Library Search API.	Disponibilidade, Fiabilidade, Desempenho	Implica criar uma camada de *gateways/adapters* com tratamento de falhas e *timeouts*, podendo usar mecanismos como *retry policy* e *circuit breaker*.	Alta
ASR-04	Funcional	Geração configurável de identificadores (IDs)	O sistema deve gerar IDs para as entidades em diferentes formatos, definidos por configuração.	Configurabilidade, Extensibilidade	Requer abstração da lógica de geração de IDs (ex.: padrão *Factory* ou *Strategy*) e suporte a configuração externa.	Média
ASR-05	Não Funcional	Extensibilidade da arquitetura	O sistema deve ser facilmente extensível a novos serviços, repositórios e integrações externas.	Extensibilidade, Evolutividade	Implica modularização clara, uso de interfaces bem definidas e inversão de dependências (*Dependency Injection* /	Alta

ID	Tipo	Requisito	Descrição	Atributo de Qualidade Afetado	Racional / Impacto Arquitetural	Prioridade
					Inversion of Control).	
ASR-06	Não Funcional	Fiabilidade e tolerância a falhas	O sistema deve continuar operacional mesmo quando uma API externa ou base de dados estiver temporariamente indisponível.	Fiabilidade, Disponibilidade	Necessário implementar mecanismos de *fallback*, *retry policies* e utilização de *cache* (ex.: Redis) para mitigar falhas temporárias.	Alta
ASR-07	Não Funcional	Configuração centralizada	O sistema deve permitir configurar integrações e persistência através de ficheiros externos (.yaml, .json, .env) ou variáveis de ambiente.	Configurabilidade, Manutenibilidade	Requer suporte a gestão centralizada de configuração (*configuration management*) e separação clara entre código e configuração.	Alta
ASR-08	Não Funcional	Melhoria dos testes automatizados	O sistema deve suportar testes automatizados em vários níveis (unitário, integração, mutação e sistema).	Testabilidade, Fiabilidade	Implica definição de fronteiras claras entre camadas, uso de *mocks* e isolamento de testes (por exemplo, através de *test containers*).	Média
ASR-09	Não Funcional	Monitorização e observabilidade	O sistema deve permitir medir desempenho e detetar falhas, suportando logs estruturados e métricas.	Fiabilidade, Manutenibilidade	Requer integração de um *framework* de logging.	Média

Adoção do ADD (Attribute-Driven Design)

Embora o projeto base já fornecesse uma estrutura monolítica em Spring Boot, a adoção do método **Attribute-Driven Design (ADD)** permitiu alinhar a arquitetura às necessidades específicas do domínio, garantindo que os requisitos de qualidade fossem sistematicamente incorporados e documentados.

O projeto aplica o método **Attribute-Driven Design (ADD)** para orientar as decisões arquiteturais.

O ADD é utilizado para transformar sistematicamente os **requisitos arquiteturalmente significativos (ASR)** em estruturas arquiteturais, focando atributos de qualidade como **extensibilidade**, **configurabilidade** e **fiabilidade**.

Processo ADD Aplicado

1. Identificação dos Requisitos Arquiteturais
   - **Funcionais:**
     - Suporte a múltiplos modelos de persistência: SQL+Redis e MongoDB+Redis.
     - Integração com APIs externas: Open Library Search API e Google Books API.

    - Geração configurável de identificadores para entidades.
- **Não Funcionais:**
    - Extensibilidade, Configurabilidade, Fiabilidade, Testabilidade.
- **Restrições Arquiteturais:**
    - A aplicação base foi fornecida como monólito RESTful em Spring Boot.
    - As configurações devem ser alteradas apenas antes do arranque da aplicação, garantindo previsibilidade e estabilidade.

2. **Seleção dos Drivers Arquiteturais**
    - Atributos de qualidade dominantes: extensibilidade, configurabilidade, fiabilidade.
    - Drivers secundários: testabilidade e desempenho.
3. **Decomposição do Sistema**
   Um exemplo concreto do impacto do ADD foi a introdução de múltiplas implementações de persistência (SQL/Mongo) e de serviços de ISBN. Estas decisões foram guiadas pelos atributos de qualidade identificados no ADD, e não faziam parte do projeto base original.
    - **Camada de Dados:** múltiplas implementações de repositórios para diferentes SGBDs.
    - **Camada de Integração:** adaptadores para APIs externas de ISBN.
    - **Camada de Configuração:** uso de *Spring Profiles* e ficheiros .
4. **Seleção de Conceitos de Design**

   Para fundamentar as decisões arquiteturais tomadas ao longo do projeto, utilizei o processo de **Quality Attribute Scenarios (QAS)**, integrado no método ADD.

   Este processo permitiu identificar, para cada atributo de qualidade relevante, situações concretas (estímulos) que o sistema deve suportar, os artefactos afetados, a resposta esperada e a métrica de sucesso.

   Com base nestes cenários, foram selecionadas e aplicadas táticas e padrões arquiteturais específicos, garantindo que cada requisito de qualidade fosse endereçado de forma explícita e rastreável na arquitetura.

Análise de Fatores Técnicos e Estratégias de Implementação

- **Variabilidade – Base de Dados**
    - **Technical Memo:** A arquitetura deve permitir alternar entre diferentes mecanismos de persistência (SQL, MongoDB) sem alterações no código de negócio, suportando requisitos de clientes ou restrições técnicas.
    - **Estímulo:**Alteração do mecanismo de BD.
    - **Artefacto:** Camada de Dados.
    - **Resposta:**Seleção do mecanismo de persistência via configuração por Spring Profile.
    - **Medida de Resposta:**Mudança sem alteração de código.
    - **Patterns Aplicados:** Strategy, Abstract Factory, Dependency Injection.
    - **Táticas ADD Aplicadas:** Encapsulate, Defer Binding, Restrict Dependencies.
    - **Alternativas Analisadas:** Configuração através de ficheiro de propriedades. Esta alternativa poderia trazer mais flexibilidade, principalmente se o objetivo fosse a alteração do comportamento de persistência da aplicação em runtime, ao expor essas propriedades, por exemplo, por JMX. No entanto, já que consideramos a camada de persistência algo mais estável, optamos pela alternativa dos Profiles, que instancia apenas os Beans necessários durante o startup da aplicação.
- **Variabilidade – API ISBN**
    - **Technical Memo:**O sistema deve permitir alternar entre diferentes APIs externas de ISBN (OpenLibraryApi, Google Books) de forma transparente, garantindo continuidade do serviço em caso de falha ou alteração de requisitos.
    - **Estímulo:** Alteração ou falha da API ISBN.

- **Artefacto:** Serviço de ISBN.
- **Resposta:** Seleção automática da API ativa ou fallback em caso de falha.
- **Medida de Resposta:** Serviço disponível, erro registado.
- **Patterns Aplicados:** Adapter, Strategy, Retry Policy.
- **Táticas ADD Aplicadas:** Encapsulate, Use an Intermediary, Maintain multiple copies of computations.

- **Variabilidade – Gerador de ID**
  - **Technical Memo:**Deve ser possível alternar entre diferentes algoritmos de geração de identificadores, conforme requisitos funcionais ou de desempenho, sem impactar o restante sistema.
  - **Estímulo:** Alteração do algoritmo de ID.
  - **Artefacto:** Serviço de Geração de ID.
  - **Resposta:**Seleção do algoritmo via configuração.
  - **Medida de Resposta:** Mudança sem alteração de código.
  - **Patterns Aplicados:** Strategy, Dependency Injection.
  - **Táticas ADD Aplicadas:** Encapsulate, Defer Binding.
  - **Alternativas analisadas:** Configuração através de Spring Profiles
- **Variabilidade – Base de Dados**
  - **Technical Memo:** A arquitetura deve permitir alternar entre diferentes mecanismos de persistência (SQL, MongoDB) sem alterações no código de negócio, suportando requisitos de clientes ou restrições técnicas.
  - **Estímulo:**Alteração do mecanismo de BD.
  - **Artefacto:**Camada de Dados.
  - **Resposta:**Seleção do mecanismo de persistência via configuração (Spring Profiles).
  - **Medida de Resposta:**Mudança sem alteração de código.
  - **Patterns Aplicados:** Strategy, Abstract Factory, Dependency Injection.
  - **Táticas ADD Aplicadas:** Encapsulate, Defer Binding, Restrict Dependencies.
  - **Alternativas Analisadas:** Configuração através de ficheiro de configuração
- **Extensibilidade**
  - **Technical Memo:**A arquitetura deve permitir a adição de novos mecanismos de persistência (SGBDs), APIs externas ou algoritmos (ex: geração de ID) sem introduzir breaking changes no código existente, facilitando a evolução e manutenção do sistema.
  - **Estímulo:**Requisito de um novo mecanismo (ex: novo SGBD, nova API, novo algoritmo).
  - **Artefacto:**Camada de Dados, Serviços.
  - **Resposta:**Adição de nova implementação sem necessidade de alterar código existente.
  - **Medida de Resposta:**Novo mecanismo integrado sem breaking changes.
  - **Patterns Aplicados:** Modularização, Abstração, Dependency Injection.
  - **Táticas ADD Aplicadas:** Split Module, Encapsulate, Restrict Dependencies.
  - **Alternativas analisadas:** Uma alternativa seria uma arquitetura monolítica rígida, onde a introdução de novos mecanismos exigiria alterações profundas no código existente. Outra alternativa seria a adoção de microserviços, o que aumentaria a complexidade e os requisitos de infraestrutura. A solução adotada foi uma arquitetura modular dentro do monólito, permitindo extensibilidade sem comprometer a simplicidade.
- **Configurabilidade**
  - **Technical Memo:**A troca de mecanismos (base de dados, API externa, algoritmo de ID) deve ser realizada exclusivamente através de configuração, sem necessidade de alterar ou recompilar o código, promovendo flexibilidade e facilidade de operação.

- **Estímulo:**Alteração de configuração.
- **Artefacto:**Configuração.
- **Resposta:**Seleção automática do mecanismo pretendido via profile ou ficheiro de configuração.
- **Medida de Resposta:**Troca efetuada sem alteração de código.
- **Patterns Aplicados:** Spring Profiles, Abstract Factory.
- **Táticas ADD Aplicadas:** Defer Binding.
- **Alternativas analisadas:** Uma alternativa seria exigir alterações de código e recompilação sempre que fosse necessário trocar de mecanismo, o que reduziria a flexibilidade e aumentaria o tempo de manutenção. Outra alternativa seria permitir alterações em runtime, mas isso poderia introduzir instabilidade e complexidade desnecessária. A opção por configuração via profiles e ficheiros externos oferece um equilíbrio entre flexibilidade e robustez.

- **Fiabilidade**
  - **Technical Memo:**O sistema deve garantir disponibilidade e continuidade de serviço perante falhas/limitações de APIs externas, recorrendo a mecanismos de fallback e registo de incidentes para evitar interrupções.
  - **Estímulo:** Falha de API externa.
  - **Artefacto:** Serviço de ISBN.
  - **Resposta:** Caching e Fallback automático para API alternativa e registo do incidente.
  - **Medida de Resposta:** Serviço disponível sem interrupção; erro devidamente registado.
  - **Patterns Aplicados:** Chain of Responsibility, Facade.
  - **Táticas ADD Aplicadas:** Maintain multiple copies of computations, Use an Intermediary.
  - **Alternativas Analisadas:**
    - Retry em caso de falha - seria uma alternativa para o caso de falha temporária de uma das APIs, mas caso a API estiver em falha por tempo indeterminado ou não contiver a informação que pretendemos a solução introduziria apenas mais latência na nossa aplicação.
    - Circuit Breaker - Interrupção temporária das chamadas às APIs instáveis, com reativação automática após período de estabilidade.

- **Testabilidade**
  - **Technical Memo:**A arquitetura deve facilitar a realização de testes isolados sobre integrações externas, permitindo a substituição por mocks ou stubs durante o desenvolvimento e testes automatizados.
  - **Estímulo:**Criação de novo teste automatizado.
  - **Artefacto:**Serviços externos.
  - **Resposta:**Possibilidade de mockar dependências externas.
  - **Medida de Resposta:**Testes isolados e repetíveis.
  - **Patterns Aplicados:** Mocking, Dependency Injection.
  - **Táticas ADD Aplicadas:** Encapsulate, Restrict Dependencies.
  - **Alternativas analisadas:** Uma alternativa seria depender sempre de integrações reais durante os testes, tornando-os mais lentos e frágeis. Outra alternativa seria criar stubs manuais para cada dependência, aumentando o esforço de manutenção. A opção por mocking e injeção de dependências permite maior flexibilidade e automatização dos testes.

- **Desempenho**
  - **Technical Memo:**O sistema implementa mecanismos de cache (Redis) tanto para dados persistentes (armazenados nas SGBDs) como para resultados de pesquisas de ISBN. Isto permite minimizar a latência em consultas repetidas, acelerando as respostas e reduzindo a carga sobre as APIs externas e as bases de dados.

    - **Estímulo:**Consulta repetida de dados persistentes (ex: livros, utilizadores) ou de ISBN.
    - **Artefacto:**Camada de Dados (SGBDs/Redis) e Serviço de ISBN/Cache.
    - **Resposta:**As respostas a consultas repetidas são servidas diretamente do Redis (cache hit), evitando acessos desnecessários às SGBDs ou às APIs externas.
    - **Medida de Resposta:**Tempo de resposta inferior a 100ms para cache hit; redução significativa de chamadas externas.
    - **Patterns Aplicados:** Cache Redis.
    - **Táticas ADD Aplicadas:** Maintain multiple copies of data, Reduce overhead.
    - **Alternativas analisadas:** Uma alternativa seria não utilizar qualquer mecanismo de cache, o que resultaria em maior latência e sobrecarga das bases de dados e APIs externas. Outra alternativa seria implementar cache apenas em memória local, o que não seria escalável em ambientes distribuídos. A opção por Redis permite partilhar a cache entre múltiplas instâncias da aplicação, garantindo desempenho e escalabilidade.
- **Evolutividade**
    - **Technical Memo:** A arquitetura deve permitir a evolução fácil do sistema, suportando a introdução de novas funcionalidades, alterações de requisitos e integração de novas tecnologias com impacto mínimo no código existente.
    - **Estímulo:** Introdução de um novo serviço ou alteração significativa de requisitos.
    - **Artefacto:** Camadas de Serviços e Dados.
    - **Resposta:** Adição ou alteração realizada com impacto mínimo e sem breaking changes.
    - **Medida de Resposta:** Tempo e esforço reduzidos para evoluir o sistema.
    - **Patterns Aplicados:** Modularização, Abstração, Dependency Injection.
    - **Táticas ADD Aplicadas:** Split Module, Encapsulate, Restrict Dependencies.
    - **Alternativas analisadas:** Uma alternativa seria adotar uma arquitetura monolítica rígida, com forte acoplamento entre componentes, o que dificultaria a evolução e introdução de novas funcionalidades. Outra alternativa seria a adoção de microserviços desde o início, o que aumentaria a complexidade e os requisitos de infraestrutura, não sendo compatível com as restrições do projeto base. Optou-se por uma arquitetura modular dentro do monólito, equilibrando simplicidade, flexibilidade e facilidade de evolução.

Explicação das Táticas e Padrões Arquiteturais Aplicados

As táticas e padrões arquiteturais selecionados foram fundamentais para garantir que cada fator técnico fosse devidamente suportado pela arquitetura. Eis um resumo das principais estratégias adotadas:

- **Strategy Pattern:**

  Permite alternar dinamicamente entre diferentes implementações de um mesmo serviço (por exemplo, diferentes mecanismos de persistência ou algoritmos de geração de IDs), promovendo variabilidade e extensibilidade sem necessidade de alterar o código de negócio.

- **Abstract Factory:**

  Facilita a criação de famílias de objetos relacionados (ex: repositórios para diferentes SGBDs) com base na configuração ativa, promovendo configurabilidade e desacoplamento.

- **Adapter Pattern:**

  Abstrai as integrações com APIs externas, permitindo que o sistema comunique com diferentes fornecedores (ex: Open Library Search API e Google Books API) de forma transparente e facilmente substituível.

- **Repository Pattern:**

  Isola a lógica de acesso e manipulação de dados da lógica de negócio da aplicação. Desta forma, o domínio não precisa conhecer os detalhes sobre a base de dados ou o mecanismo de persistência utilizado, promovendo testabilidade e extensibilidade.

- **Dependency Injection (IoC):**

Permite a injeção de dependências em tempo de execução, ativando componentes e serviços de acordo com a configuração definida, promovendo modularidade, testabilidade e flexibilidade.

- **Chain of Responsibility (CoR):**

  Encadeia vários handlers para um mesmo pedido, em que cada handler tenta trata-lo de forma independente.

  Caso um handler não consiga processar o pedido, este é passado para o próximo na cadeia.

- **Retry Policy & Circuit Breaker:**

  Garantem fiabilidade ao lidar com falhas temporárias em APIs externas, permitindo tentativas automáticas e fallback para alternativas, assegurando continuidade do serviço.

- **Mocking:**

  Facilita a substituição de dependências externas por versões simuladas durante testes, promovendo testabilidade e isolamento.

- **Cache Redis:**

  Utilizado para melhorar o desempenho, reduzindo a latência em consultas repetidas e minimizando o número de chamadas a serviços externos.

Explicação das Táticas ADD Aplicadas

As táticas arquiteturais do ADD foram selecionadas para garantir que os atributos de qualidade identificados fossem efetivamente suportados pela arquitetura. Eis um resumo das principais táticas aplicadas:

- **Encapsulate:**

  Isola detalhes de implementação atrás de interfaces ou classes abstratas, reduzindo o acoplamento entre módulos e facilitando a substituição ou evolução de componentes sem impactar o restante sistema.

- **Defer Binding:**

  Adia a decisão de ligação de componentes (por exemplo, qual implementação de um serviço usar) para o tempo de configuração ou execução, permitindo maior flexibilidade e adaptabilidade do sistema.

- **Restrict Dependencies:**

  Limita as dependências entre módulos, promovendo independência e facilitando a manutenção, evolução e testabilidade do sistema.

- **Use an Intermediary:**

  Introduz um componente intermediário (ex: façade, adapter) entre módulos para desacoplar interações diretas, facilitando a integração de novos serviços ou a substituição de componentes.

- **Maintain Multiple Copies of Data/Computations:**

  Mantém cópias redundantes de dados (ex: cache Redis) ou de lógica de processamento para melhorar desempenho, disponibilidade e tolerância a falhas.

- **Split Module:**

  Divide módulos grandes em componentes menores e mais coesos, facilitando a compreensão, manutenção e evolução do sistema.

- **Abstract Common Services:**

  Extrai funcionalidades comuns para serviços ou componentes partilhados, promovendo reutilização e reduzindo duplicação de código.

- **Reduce Overhead:**

  Minimiza operações desnecessárias ou repetidas, otimizando o desempenho do sistema (ex: servindo dados do cache em vez de repetir chamadas externas).

5. **Instanciação e Interfaces**
   Por exemplo, para suportar os requisitos **ASR-01** (Suporte a múltiplos modelos de persistência) e **ASR-02** (Configuração dinâmica da persistência antes da execução), foram definidas interfaces e implementações específicas:

   - `BookRepository` → `SqlBookRepository` e `MongoBookRepository`.

- `IsbnService` → `IsbnDbAdapter` e `GoogleBooksAdapter`.
- Perfis Spring (`@Profile("sql")`, `@Profile("mongo")`) determinam qual é instanciada no arranque.

6. **Desenho e Documentação**

   A arquitetura foi documentada seguindo o **modelo C4**, representando o sistema em diferentes níveis de abstração (contexto, containers, componentes e código).

   Os diagramas foram elaborados em **PlantUML**, garantindo que a documentação é versionável, automatizável e facilmente atualizável.

Diagramas C4 incluídos:

- **Vista de Contexto (C1):**

  Mostra o sistema no seu ambiente e as interações com utilizadores e APIs externas (ISBNdb, Google Books).

- **Vista de Containers (C2):**

  Identifica os containers principais — API REST (Spring Boot), Base de Dados (PostgreSQL), Cache (Redis) e APIs externas.

- **Vista de Componentes (C3):**

  Detalha *Controllers*, *Services*, *Repositories* e *Adapters*.

- **Vista de Código (C4):**

  Mostra classes-chave e interfaces como `BookService`, `BookRepository`, `IsbnAdapter`.

- **Vista de Sequência:**

  Descreve o fluxo da "Consulta de ISBN" com cache Redis e fallback em APIs externas.

- **Vista Física:**

  Representa a distribuição física dos componentes e as comunicações entre eles (HTTP, JDBC, etc.).

1. **Análise e Refinamento**
   - Testes de integração e mutação verificam se as implementações cumprem os ASR.
   - A prioridade foi dada à simplicidade e manutenibilidade sobre a reconfiguração dinâmica em runtime.

Resumo das Táticas Arquiteturais

Atributo de Qualidade	Táticas Aplicadas
Extensibilidade	Modularização, Padrão Strategy, Inversão de Dependências
Configurabilidade	*Spring Profiles*, ficheiros YAML, Abstract Factory
Fiabilidade	Retry Policy, Circuit Breaker, uso de Redis para cache
Testabilidade	Mocking de serviços externos, injeção de dependências
Desempenho	Cache Redis, otimização de queries
Disponibilidade	Requisições redundantes a múltiplas APIs externas

---

Resumo dos Padrões de Arquitetura

- **Arquitetura em Camadas (Layered Architecture)**

  Controllers → Services → Repositories → Domain

- **Repository Pattern**

  Abstração da persistência

- **Adapter Pattern**

  Integração de serviços externos

- **Strategy Pattern**

  Seleção de implementação com base no *profile*

- **Dependency Injection (IoC)**

  Substituição dinâmica de componentes consoante a configuração

Alternativas Arquiteturais e Justificação

Alternativa	Vantagens	Desvantagens	Decisão
Configuração via Spring Profiles (build-time)	Simples, nativa no Spring, bem documentada	Requer reinício da aplicação	✅ Aceite
Configuração dinâmica em runtime (ConfigManager)	Alta flexibilidade	Complexidade desnecessária e risco de instabilidade	❌ Rejeitada
Microserviços independentes	Escalabilidade e isolamento	Violava restrição do projeto base (monólito REST)	❌ Rejeitada
Monólito modular	Simplicidade, coerente com o sistema existente	Menor isolamento	✅ Aceite

> Nota:
>
> Embora o modelo de microserviços tenha sido avaliado no processo ADD, foi rejeitado devido às **restrições do projeto base**, que exigia a manutenção de uma **arquitetura monolítica RESTful**. Assim, o foco foi otimizar a modularidade e extensibilidade **dentro do monólito**.

Arquiteturas de Referência

- Aplicações RESTful Spring Boot
- Arquitetura em camadas (Controller–Service–Repository–Domain)
- Integração com Redis e MongoDB
- Documentação com Swagger/OpenAPI
- Testes automatizados (JUnit + Mockito + Mutation Testing)

Evidência de Implementação e Testes

- **Testes funcionais (opaque-box e transparent-box)** asseguram o cumprimento dos requisitos funcionais e ASR.
- **Testes de mutação** confirmam a robustez das implementações.
- Os testes abrangem:
  - Classes individuais (opaque-box)
  - Domínio (transparent-box)
  - Integração Controller + Service + Repository
  - Teste de sistema completo (API)

Resumo Final:

O presente relatório documenta de forma sistemática o processo de análise, decisão e implementação arquitetural de um sistema de gestão de biblioteca, recorrendo ao método Attribute-Driven Design (ADD) e ao modelo C4 para documentação visual. O trabalho desenvolvido demonstra uma abordagem rigorosa à identificação e tratamento dos requisitos arquiteturalmente significativos (ASR), com especial foco nos atributos de qualidade de extensibilidade, configurabilidade, fiabilidade, testabilidade e desempenho.

A arquitetura resultante é modular, configurável e fiável, suportando múltiplos modelos de persistência (SQL, MongoDB, Redis), integração com diferentes APIs externas de ISBN e mecanismos flexíveis de geração de identificadores. O uso de padrões de design como Strategy, Adapter, Abstract Factory, Repository e Dependency Injection, aliado à aplicação de táticas ADD clássicas (Encapsulate, Defer Binding, Maintain Multiple Copies, etc.), permitiu construir uma solução robusta, facilmente evolutiva e alinhada com as melhores práticas da engenharia de software.

O processo ADD foi seguido integralmente, desde a análise dos requisitos até à justificação das alternativas arquiteturais, seleção de táticas e validação por testes automáticos. A documentação inclui diagramas C4 em vários níveis (contexto, containers, componentes, código e sequência), facilitando a compreensão global do sistema e a comunicação entre stakeholders.

Apesar das restrições impostas pelo projeto base (monólito RESTful), foi possível maximizar a modularidade e a flexibilidade, preparando a arquitetura para futuras evoluções e integrações. A evidência de implementação e testes demonstra o alinhamento entre design, código e requisitos, assegurando a qualidade e a manutenibilidade do sistema.

Em suma, este projeto constitui um exemplo prático e fundamentado de aplicação de métodos arquiteturais modernos, promovendo uma arquitetura sustentável, escalável e preparada para responder a novos desafios do domínio da gestão de bibliotecas.